National University
Of Computer and Emerging Sciences

Foundation of Advancement
Of Science and Technology

Lab Number:	10
Course Code:	CL2005
Course Title:	Database Systems - Lab
Semester:	Spring 2026
Agenda:	SQL Views, Indexing & Built-In Functions
Instructor:	Muhammad Mehdi
Email:	muhammad.mehdi@nu.edu.pk
Office:	Rafaqat Lab

Table of Contents

1 SQL Query Execution Order

The **SQL order of execution** describes the sequence in which a database processes the clauses of a query, which is often different from the order in which they are written. A solid understanding of this execution flow, also known as the **SQL order of operations**, is essential for writing correct and efficient queries.

Many common issues, such as unexpected results, errors, or poor performance, arise from misunderstandings about how SQL evaluates a query. By understanding the execution order, you can more easily troubleshoot why a query fails, predict how data will be processed, and optimize queries for better performance.

1.1 Logical Execution Order

The following table describes the logical order in which SQL query clauses are executed:

Step	Clause	Function
1	`FROM`	Identify source tables, execute subqueries, create initial dataset
2	`JOIN`	Combine tables based on join conditions
3	`WHERE`	Filter rows before grouping, evaluate row-level conditions and subqueries
4	`GROUP BY`	Group rows into aggregates based on columns
5	`HAVING`	Filter grouped data using aggregate conditions
6	`SELECT`	Compute final columns, apply aggregates, expressions, subqueries, assign aliases
7	`ORDER BY`	Sort result set using columns, aliases, or expressions
8	`LIMIT / OFFSET`	Restrict and/or skip number of rows returned

1.2 Example

Find the 2nd and 3rd highest departments (by employee count) where more than two employees earn above average salary.

```
SELECT d.name AS department_name, COUNT(*) AS total
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE salary > (SELECT AVG(salary) FROM employees)
GROUP BY d.name
HAVING COUNT(*) > 2
ORDER BY total DESC
LIMIT 2
OFFSET 1;
```

Explanation:

The above query retrieves the **top departments (excluding the first highest)** that have more than two employees earning above the company's average salary.

- Joins the `employees` and `departments` tables to associate employees with their department names
- Filters employees whose salary is greater than the overall average salary (using a subquery)
- Groups the remaining employees by department
- Keeps only those departments with more than 2 such employees
- Counts the number of qualifying employees in each department
- Sorts the departments by this count in descending order
- Skips the top result (`OFFSET 1`) and returns the next 2 departments (`LIMIT 2`)

2 SQL Views

Views are **virtual tables** that are based on the result of a query. They do **not store data physically** but only the query definition is stored. A **view** behaves like a table in queries. A view always reflects the **latest data** from its underlying tables, which are called **base tables**.

2.1 Types of Views

2.1.1 Updatable Views

- Based on a **single table**
- No aggregates, joins, grouping, or computed columns
- Allows **DML** operations
- Can use `WITH CHECK OPTION` to enforce filter conditions

2.1.2 Non-Updatable Views

- Based on **complex queries** (joins, aggregates, grouping, computed columns etc.)
- DML operations are **not allowed**
- Cannot enforce filter conditions

2.2 Why Use Views?

- Simplify complex or frequently used queries
- Improve security by hiding sensitive columns
- Promote reusability of SQL logic
- Provide an abstraction layer over base tables

2.3 Creating a View

General Syntax:

```
CREATE VIEW view_name AS
SELECT column(s)
FROM table(s)
WHERE condition
...;

CREATE OR REPLACE VIEW view_name AS
SELECT column(s)
FROM table(s)
WHERE condition
...;
```

Examples:

```
CREATE VIEW high_salary_employees AS
SELECT name, salary
FROM employees
WHERE salary > 6500;

CREATE OR REPLACE VIEW it_employees AS
SELECT id, name, salary, department_id
FROM employees e
WHERE department_id = (
    SELECT id FROM departments WHERE name = 'IT'
);
```

2.4 Using a View

Once a view is created, you can query it just like a table:

```
SELECT * FROM high_salary_employees;

SELECT name, salary FROM it_employees;
```

2.5 Updating a View

2.5.1 Performing DML on Updatable Views

DML (INSERT, UPDATE, DELETE) operations can be performed on updatable views to make changes to the underlying base table(s).

```
-- Insert via view
INSERT INTO it_employees (id, name, salary, department_id)
VALUES (NULL, 'Hasnain', 4300, 4);

-- Update via view
UPDATE it_employees
SET department_id = 1
WHERE name LIKE 'Hasnain';

-- Delete via view
DELETE FROM it_employees
```

```
WHERE name LIKE 'Bilal';
```

2.5.2 Enforcing Filter Conditions with Views

SQL provides a command of `WITH CHECK OPTION` which is used to ensure that any `INSERT` **or** `UPDATE` through the view **cannot violate the view's filter condition**.

It ensures **data integrity** according to the view's `WHERE` clause.

Example: Updatable View with Filter

```
/* Every row must always have department_id = 4 (IT) when
inserted or updated through the view */

CREATE OR REPLACE VIEW it_employees AS
SELECT id, name, salary, department_id
FROM employees e
WHERE department_id = (
    SELECT id FROM departments WHERE name = 'IT'
) WITH CHECK OPTION;
```

Allowed & Restricted Operations:

```
-- Allowed/Successful insertion
INSERT INTO it_employees (id, name, salary, department_id)
VALUES (NULL, 'Faraz', 5000, 4);

-- Restricted/Failed insertion
INSERT INTO it_employees (id, name, salary, department_id)
VALUES (NULL, 'Hasnain', 7000, 1);


-- Allowed/Successful update
UPDATE it_employees
SET salary = 3000
WHERE name LIKE 'Faraz';

-- Restricted/Failed update
UPDATE it_employees
```

```
SET department_id = 2
WHERE name LIKE 'Faraz';
```

2.6 Dropping a View

General Syntax:

```
DROP VIEW view_name;
```

Examples:

```
DROP VIEW high_salary_employees;

DROP VIEW it_employees;
```

3 Indexing

Indexing is a technique/mechanism that **speeds up data retrieval** from a dataset. They work like the index of a book, pointing to the location of the data, which removes the need to scan the entire dataset.

3.1 SQL Indexing

- An index in SQL stores a **sorted representation** of one or more columns.
- Indexes are applied to **columns** in a table, and are especially useful for `SELECT` **queries, filtering, and joins**.
- Instead of scanning all rows, the database server can **jump directly** to the relevant row(s).
- Indexes are transparent to users, queries automatically use them if beneficial.

3.2 Benefits & Drawbacks

Benefits:

- Faster search in large tables
- Faster filtering using `WHERE` conditions
- Faster join operations

Drawbacks:

- Slower INSERT, UPDATE, DELETE operations (indexes need to be maintained)
- Extra storage space required for the index

3.3 Creating an Index

General Syntax:

```
CREATE INDEX index_name
ON table_name (column_name);

-- OR

ALTER TABLE table_name
ADD INDEX index_name (column_name);
```

Examples:

```
CREATE INDEX idx_salary
ON employees (salary);

ALTER TABLE students
ADD INDEX idx_std_name (name);

-- Composite Index
CREATE INDEX idx_dept_salary
ON employees (department_id, salary);
```

3.4 Dropping an Index

General Syntax:

```
DROP INDEX index_name ON table_name;

-- OR

ALTER TABLE table_name
DROP INDEX index_name;
```

Examples:

```
DROP INDEX idx_salary ON employees;

-- OR

ALTER TABLE employees
DROP INDEX idx_dept_salary;
```

4 Built-in Functions in MySQL

MySQL provides built-in functions that allow you to perform calculations, transformations, and data manipulation directly within SQL queries. These functions help reduce application-side processing and make queries more powerful and efficient.

Many MySQL functions have **equivalents in other database management systems** (syntax may vary slightly)

Functions can take as input:

- **Constants or literals** (e.g., `123`, `'2026-04-01'`)
- **Columns/attributes** from tables (e.g., `salary`, `hire_date`)

Therefore, functions can appear **anywhere a value or expression is allowed** in a query.

4.1 Numeric Functions

Numeric functions are used to perform **mathematical operations** on numeric data.

Function	Description
`ROUND(number, decimals)`	Rounds a number to the specified number of decimal places
`CEIL(number)`	Rounds a number up to the nearest integer
`FLOOR(number)`	Rounds a number down to the nearest integer
`ABS(number)`	Returns the absolute (non-negative) value of a number

Examples:

```
SELECT ROUND(123.456, 2) AS rounded_value;
SELECT CEIL(123.1) AS ceil_value;
SELECT FLOOR(123.9) AS floor_value;
SELECT ABS(-50) AS absolute_value;

/* Show each student's CGPA rounded to 1 decimal and difference
from perfect CGPA (4.0) */
SELECT
    name,
    ROUND(cgpa, 1) AS rounded_cgpa,
    ABS(4.0 - cgpa) AS gap_from_perfect
FROM students;
```

4.2 String Functions

These functions are used to **manipulate and transform text data**. They are commonly used for formatting output, cleaning data, and extracting specific parts of strings.

Function	Description
`CONCAT(str1, str2, ...)`	Joins two or more strings into a single string
`LENGTH(string)`	Returns the length of a string in bytes
`UPPER(string)`	Converts all characters in a string to uppercase
`LOWER(string)`	Converts all characters in a string to lowercase
`SUBSTRING(string, start, length)`	Extracts a substring from a string starting at a given position

Examples:

```
SELECT CONCAT('Hello', ' ', 'World') AS result;
SELECT LENGTH('Database') AS length_value;
SELECT UPPER('sql lab') AS upper_case;
SELECT LOWER('SQL LAB') AS lower_case;
SELECT SUBSTRING('Database', 1, 4) AS sub; -- Index starts at 1

/* Display formatted student info with uppercase names and
department labels */
```

```
SELECT
    CONCAT(UPPER(s.name), ' - ', LOWER(d.name)) AS student_info,
    LENGTH(s.name) AS name_length
FROM students s
JOIN departments d ON s.department_id = d.id;
```

4.3 Type Conversion Functions

These functions are used to **convert data from one datatype to another**. In MySQL, some conversions occur implicitly. For example, MySQL automatically converts numbers to strings when needed, and vice versa.

Common target datatypes:

- `CHAR`, `SIGNED`, `UNSIGNED`
- `DECIMAL(p, s)`
- `DATE`, `DATETIME`

Function	Description
`CAST(expression AS datatype)`	Converts a value to a specified target datatype using standard SQL syntax
`CONVERT(expression, datatype)`	Converts a value to a specified target datatype (MySQL-specific syntax)

Examples:

```
SELECT 10 + '10' AS converted_value; -- Implicit conversion
SELECT CAST('123.45' AS DECIMAL(5,2)) AS converted_value;
SELECT CONVERT('123', SIGNED) AS converted_value;

/* Convert salary to integer and compare with department budget
portion */
SELECT
    name,
    salary,
    CAST(salary AS SIGNED) AS salary_int,
    CONVERT(salary / 1000, DECIMAL(5,2)) AS salary_in_thousands
FROM employees;
```

4.4 Conditional Functions

These functions are used to **apply logic within queries**, handle missing values, and return results based on conditions. These are especially useful for **data cleaning, formatting, and decision-making directly in SQL**.

Function	Description
`IFNULL(expression, replacement)`	Returns a replacement value if the expression is NULL
`COALESCE(expr1, expr2, ...)`	Returns the first non-NULL value from the list of expressions

Examples:

```
SELECT IFNULL(NULL, 'No Value') AS result;
SELECT COALESCE(NULL, NULL, 'First', 'Another') AS result;

/* Replace NULL salaries (if any) and categorize employees by
salary level */
SELECT
    name,
    IFNULL(salary, 0) AS safe_salary,
    COALESCE(
        CASE
            WHEN salary >= 6500 THEN 'High'
            WHEN salary >= 5000 THEN 'Medium'
        END,
    'Low') AS salary_category
FROM employees;
```

5 Working with Temporal Data

Date and time values in MySQL are stored using specialized data types such as:

Datatype	Format	Example
DATE	*'YYYY-MM-DD'*	'2026-04-01'
TIME	*'HH:MM:SS'*	'15:30:45'

`DATETIME / TIMESTAMP`	*'YYYY-MM-DD HH:MM:SS'*	'2026-04-01 15:30:45'

Although they are internally stored in a compact binary format, they are typically **displayed and written as a string in standard ISO format**.

The ISO format:

- Sorts correctly (chronological order)
- Is easy to read and understand
- Is consistent across most database systems

Because of this standardized format, SQL allows **direct comparison** using normal comparison operators:

`=, <>, <, >, <=, >=`

Example:

```
-- Find all courses enrolled done after February 1, 2026
SELECT *
FROM enrollments
WHERE enrollment_date > '2026-02-01';
```

For more advanced operations (formatting, extraction, arithmetic), SQL provides **built-in date & time functions**.

5.1 Current Date & Time Functions

These functions return the **current system date and/or time** based on the database server.

Function	Description
`CURRENT_TIMESTAMP` / `NOW()`	Returns current date and time
`CURRENT_DATE` / `CURDATE()`	Returns current date
`CURRENT_TIME` / `CURTIME()`	Returns current time

Examples:

```
SELECT CURRENT_TIMESTAMP AS current_datetime;
SELECT CURRENT_DATE AS "current_date";
```

```
SELECT CURRENT_TIME AS current_time;
```

5.2 Date Extraction Functions

These functions are used to **extract specific parts (components)** from date and/or time values.

Function	Description
`YEAR(date)`	Extracts year from date
`MONTH(date)`	Extracts month (1-12)
`DAY(date)`	Extracts day of the month (1-31)
`LAST_DAY(date)`	Returns last date of the month

Examples:

```
SELECT YEAR(NOW()) AS year;
SELECT MONTH(NOW()) AS month;
SELECT DAY(NOW()) AS day;
SELECT LAST_DAY(NOW()) AS last_day;

/* Extract year and month of enrollments to analyze when students
enroll most */
SELECT
    student_id,
    YEAR(enrollment_date) AS year,
    MONTH(enrollment_date) AS month,
    DAY(enrollment_date) AS day
FROM enrollments;
```

5.3 Date Formatting

In SQL, date formatting is used to:

- Display dates in a readable format **(output)**
- Convert formatted strings into valid `DATE`/`DATETIME` types/values **(input)**

Function	Description

`DATE_FORMAT(date, format)`	Converts a date into a formatted string
`STR_TO_DATE(string, format)`	Converts a string into a date or datetime

Common Format Specifiers:

Specifier	Description
Day	
`%d`	Day of month (01-31)
`%D`	Day of month with suffix (1st-31st)
`%j`	Day of year (001-366)
`%W`	Weekday name (Sunday-Saturday)
`%a`	Short weekday name (Sun-Sat)
Month	
`%m`	Month (01-12)
`%M`	Full month name (January-December)
`%b`	Short month name (Jan-Dec)
Year	
`%Y`	4-digit year
`%y`	2-digit year
Time	
`%H`	Hour (00-23)
`%h / %I`	Hour (01-12)
`%i`	Minutes, numeric (00-59)
`%s`	Seconds (00-59)
`%f`	Microseconds

%p	AM/PM
Combined Time	
%T	24-hour time (hh:mm:ss)
%r	12-hour time (hh:mm:ss AM/PM)

Examples:

```
-- Formatting Output
SELECT DATE_FORMAT(NOW(), '%d-%m-%Y') AS common_format;
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y') AS european_format;
SELECT DATE_FORMAT(NOW(), '%W, %M %d, %Y') AS readable_format;
SELECT DATE_FORMAT(NOW(), '%d-%m-%Y %H:%i:%s') AS iso_format;
SELECT DATE_FORMAT(NOW(), '%h:%i %p') AS 12_hour_format;

-- Convert string to DATE
INSERT INTO employees (hire_date)
VALUES (STR_TO_DATE('01/04/2026', '%d/%m/%Y'));

-- Convert string to TIME
INSERT INTO orders (order_time)
VALUES (STR_TO_DATE('3:10:45 PM', '%h:%i:%s %p'));

-- Show converted non-standard string to standard type
SELECT STR_TO_DATE('4,1,2025 14:5', '%m,%d,%Y %H:%i') AS test;

/* Display enrollment dates in a readable format for reports */
SELECT
    student_id,
    DATE_FORMAT(enrollment_date, '%W, %M %d, %Y') AS format_date
FROM enrollments;
```

5.4 Date & Time Arithmetic Functions

These functions are used to **add, subtract, or compare dates and times** using intervals.

In MySQL, time intervals are specified using the `INTERVAL` keyword.

General Syntax:

```
INTERVAL value unit
```

Where:

- **value:** Any integer
- **unit:** One of available units in MySQL including DAY, MONTH, YEAR, HOUR, MINUTE, SECOND etc.

Function	Description
DATE_ADD(date, INTERVAL value unit)	Adds time interval to date
DATE_SUB(date, INTERVAL value unit)	Subtracts time interval from date
DATEDIFF(date1, date2)	Returns difference in days
TIMEDIFF(time1, time2)	Returns difference in hours, minutes and seconds
TIMESTAMPDIFF(unit, datetime1, datetime2)	Returns difference in any unit (seconds, days etc.)

Examples:

```
-- Add 10 days
SELECT DATE_ADD(CURDATE(), INTERVAL 10 DAY);

-- Subtract 1 month
SELECT DATE_SUB(NOW(), INTERVAL 1 MONTH);

-- Difference between two dates
SELECT DATEDIFF('2026-12-31', '2026-01-01') AS days_diff;

-- Difference between two time values
SELECT TIMEDIFF('18:00:00', '09:15:30') AS diff_time;

-- Difference in seconds between two datetimes
SELECT TIMESTAMPDIFF(SECOND, '2026-04-01 10:00:00', '2026-04-01
10:01:30') AS seconds_diff;

/* Calculate how many days have passed since each enrollment */
SELECT
```

```
    student_id,
    enrollment_date,
    DATEDIFF(CURDATE(), enrollment_date) AS days_since_enrollment
FROM enrollments;
```